Bluetooth Device Locator

Code Analysis, Bugs, and Accuracy

Findings report - React + Capacitor + BLE

Scope of review

Reviewed the core accuracy and motion logic of the project: src/lib/signal.ts, bluetoothService.ts, nameResolver.ts, useScanner.ts, useDeadReckoning.ts, breadcrumbs.ts, useTrail.ts, useClicker.ts, clickInterval.ts, useWakeLock.ts, proximityHaptics.ts, the screens, App.tsx, and the offline check scripts/simulate-signal.ts.

Baseline checks (before any change):

```
npx tsc --noEmit -p .   ->  clean
npm run sim             ->  steady ~93-96%, no wrong direction, truth in range ~96-100%
```

Contents

1. Bugs found 5

2. How to increase accuracy 6

3. What is already working well 7

1. Bugs found

1.1 Anti-deafness restart can be defeated by invalid packets

The scan-health feature that restarts a long-running BLE scan when it goes quiet is the app's most important verified behaviour. Its trigger is timestamped before the RSSI validity check, so packets that are later discarded still reset the "last heard" timer.

```
// src/lib/bluetoothService.ts:88-91
const onScan = (result: ScanResult) => {
  lastResultAt = Date.now();              // <- set BEFORE the RSSI check
  if (typeof result.rssi !== 'number' || result.rssi >= 0) return;
  // ...
};
```

If the radio keeps delivering invalid-RSSI packets (127 / 0, which a degrading scan tends to do), the 4 s silence detector never trips and the verified early restart never fires. Fix: stamp lastResultAt after the RSSI validity check, or track only valid results.

1.2 valuesSince can read far outside its window when packets are sparse

```
// src/lib/signal.ts:180-188
private valuesSince(from: number, minCount: number): number[] {
  const out: number[] = [];
  for (let i = this.raw.length - 1; i >= 0; i--) {
    const s = this.raw[i];
    if (s.ts < from && out.length >= minCount) break;
    out.push(s.rssi);
  }
  return out;
}
```

The minCount guard only breaks early; it never excludes old samples. When fewer than minCount samples fall inside the window, the loop keeps walking backwards across arbitrarily old data. The upper-quartile pre-filter and the MAD noise estimate (noiseSigma) then mix very stale and fresh readings right after a device goes quiet and returns, feeding bad z and sigma into the Kalman update. Affects low packet-rate devices most.

1.3 Apple iBeacon reference-power lookup is key-inconsistent

```
// src/lib/nameResolver.ts:358
const apple = result.manufacturerData?.['76'];
// vs extractCompanyId (line 110-123) which handles BOTH '76' and '0x004c' keys
// and decodeApplePayload which uses Object.values(manufacturerData)[0]
```

extractReferencePower reads the key '76' specifically, while the rest of the file tolerates both '76' and '0x004c' key styles. If the plugin reports the key as '0x004c', the iBeacon TX-power calibration is silently dropped and the device falls back to the default -59 dBm. Make the key lookup match the decoder's handling.

1.4 Colour and progress report raw dBm, not closeness

In SurveyScreen the list colour, signal meter and percentage are computed from raw filtered dBm (getSignalColor, SurveyScreen.tsx:100-104), while HuntScreen colours by proximity (normalised by the device's 1 m reference). Result: a loud device far away renders "green / strong" and a quiet device in your pocket renders "red / weak" - the opposite of the app's core promise ("distance, not raw dBm"). Normalise the list the same way HuntScreen does.

1.5 Trail estimate ignores distance entirely

The breadcrumb estimator weights marks by amplitude ratio only and throws away the RSSI-to-distance each mark implies (see section 2.1). This is the largest accuracy gap and is treated as a design improvement rather than a crash-level bug.

VERIFICATION NOTES
These are findings from static analysis. Bug 1.1 and 1.2 are small, safe fixes. Bugs 1.3 and 1.4 depend on real advertisement payloads and should be confirmed on a physical Android phone. Changes to signal.ts or breadcrumbs.ts
must re-run "npm run sim" before and after and keep wrong-direction rate <= ~2% and stationary steady >= ~75%.

2. How to increase accuracy

2.1 Real multilateration instead of amplitude-weighted centroid

Currently estimate() (breadcrumbs.ts:57-90) weights each crumb by amplitude ratio and averages positions. It never uses the fact that each crumb also implies a distance ring the target must lie on. Store the estimated distance (computed from the crumb's rssi plus that device's refPower) in each Crumb, then solve a least-squares "intersect the circles" position. This turns the walked loop into genuine triangulation and is the single biggest directional-accuracy win.

2.2 Pin down the path-loss exponent with a second calibration point

The path-loss exponent n is hard-coded to 2.5 (mid of 2-3) in signal.ts. Indoor clutter vs. line-of-sight changes n by roughly 1, which shifts the distance estimate by a factor of about 1.3-1.6 at mid-range. Add an optional "calibrate at ~3 m" step alongside the existing 1 m calibration and solve n from the pair. This is the biggest distance-accuracy gain available.

2.3 Feed RSSI-distance back into dead reckoning to kill drift

The integrated position (useDeadReckoning) drifts because step length and heading errors accumulate. When a crumb drops you know a distance ring around it the target must lie on; use it to periodically re-anchor the trail. This keeps the centroid / multilateration honest over long hunts.

2.4 Improve heading responsiveness

The compass smoothing alpha of 0.25 (useDeadReckoning.ts:143) is slow, so turning feels laggy and the trail's bearing estimate lags the real direction. Consider a faster alpha while turning or gyro-aided fusion; at minimum re-tune so a 90 degree turn converges in well under a second.

2.5 Reduce body-loss error

The phone's own body blocks up to ~10 dB. Partly handled in the guidance text already. An explicit "keep the phone in front of you, not in a pocket" callout at LOW confidence would remove a real source of distance error.

2.6 Make the Kalman process noise adaptive

Process noise is fixed per search mode. During fast motion you want to trust the new reading more; standing still you want to smooth more. Scaling process noise with the measured slope would tighten stationary estimates without lagging an approach.

3. What is already working well

- The trend pipeline (upper-quartile pre-filter, time-based Kalman, t-statistic gated slope with hysteresis and TREND_HOLD_MS debounce) is genuinely solid: no false direction and ~94% steady on stationary targets.

Honest distance ranges, clear calibrated / advertised / default labelling, and the "SIGNAL LOST" /

stale handling.

- Scan restart cadence and the address-rotation follower (App.tsx) are correctly designed and verified on device.
- Calibration precedence (user > advertised > default) is correct in useScanner.ts.

Recommended order of work

Highest impact on accuracy

- 2.1 Multilateration in breadcrumbs.ts (directional accuracy).
- 2.2 Per-device path-loss exponent via a second calibration point (distance accuracy).

Highest impact on robustness

- 1.1 Fix the deaf-detection timer (bluetoothService.ts).
- 1.2 Fix valuesSince window leakage (signal.ts).

Presentation consistency

- 1.4 Normalise colour / percentage by refPower in the survey list.
- 1.3 Unify the Apple manufacturerData key handling.